Large Language Models (Homework 2)

Due date : 2026/11/13 23:55 (Hard Deadline)

1 Selective Memory with State-Space Models (50%)

You will train state-space models (SSMs) from scratch to retrieve information from a sequence and participate in a Kaggle competition. The task requires the model to preserve valid key–value pairs, ignore distracting records, and predict the requested value. You will implement a basic diagonal SSM and an input-gated variant, compare them under the same conditions, and submit predictions from your best validation-selected model.



Task description:

- `STORE K_i V_j`: store a valid key–value pair.
- `NOISE K_i V_j`: ignore this distracting pair, even if the same key also appears in a valid record.
- `QUERY K_i ANSWER`: predict the value stored for this key.

For example, the following prompt requires the answer `V8`, not `V2`:

```
BOS STORE K3 V8 SEP NOISE K3 V2 SEP STORE K1 V4 SEP
NOISE K9 V5 SEP QUERY K3 ANSWER
```

Line breaks are for display only. The answer is **not** included in the input. This is restricted-vocabulary next-token prediction, not a test of general natural-language ability.

Dataset description:

File	Samples	Purpose
`data/train.jsonl`	12,000	Training; contains answers.
`data/val.jsonl`	1,500	Validation and model selection; contains answers.
`data/test.jsonl`	2,000	Kaggle test inputs; answers are hidden.
`hw2_1.ipynb`	–	Provided utilities and five model TODOs.

- Each sample contains `id` and `tokens`; labeled splits also contain `answer`. Read one JSON object per line. The short example above illustrates the format, not the training length distribution.
- A fixed 71-token vocabulary is built into the notebook: seven special tokens (`PAD, BOS, SEP, STORE, NOISE, QUERY, ANSWER`), keys `K0`–`K31`, and values `V0`–`V31`. Each whitespace-separated symbol is one token.
- Each sample has 2–4 `STORE` records with distinct keys; values may repeat. Other records are `NOISE`. Valid pairs and record order are randomized. The query key is sampled uniformly from valid stored keys; noise keys and values are sampled independently.
- Each record occupies four tokens, including `SEP`. With R records, prompt length $L = 4R + 4$. All three splits use R sampled uniformly from 6–30 (L = 28–124). Complete prompts are disjoint across splits.

Please follow the steps below to implement your program. For each question, include a short **Implementation and discussion** response in a notebook Markdown cell. Explain your own procedure and support your conclusions with the reported results; do not only paste code.

1. **Implement and train a basic diagonal SSM.** (10%)
   Let $x_t \in \mathbb{R}^d$ be a token representation and $h_t \in \mathbb{R}^N$ a hidden state. Use the following discrete-time model:

$$a = \sigma(\theta_a), \qquad h_0 = 0,$$
$$h_t = a \odot h_{t-1} + Bx_t, \qquad y_t = Ch_t + Dx_t.$$

   Here, $a \in \mathbb{R}^N$ contains the diagonal transition coefficients, $B \in \mathbb{R}^{N\times d}$, $C \in \mathbb{R}^{d\times N}$, and $D \in \mathbb{R}^{d\times d}$ are learnable. The sigmoid σ constrains a to $(0, 1)$; $\odot$ is element-wise multiplication. Do not add biases to B, C, or D. No continuous-time discretization is required.

   (a) Complete the `basic_init` and `basic_forward` TODOs in `DiagonalSSM`. Initialize parameters and implement the state update and output. (5%)

   (b) Train the model, plot training/validation loss, and report the best validation accuracy and its epoch. (3%)

   (c) **Implementation and discussion.** Explain your forward-pass procedure and the input, state, and output shapes. Describe how you reset states and select the final valid `ANSWER` position in a padded batch. Use your loss curves to discuss convergence or overfitting. (2%)

Shared backbone for Problem 1, Questions 1–3:

- Use an embedding, two causal SSM blocks with independent parameters, a final LayerNorm, and a 32-way answer-token head. Set $d = N = 64$ and dropout to 0.1.
- Each block computes

$$x_{\text{next}} = x + \text{Dropout}(\text{GELU}(\text{SSM}(\text{LayerNorm}(x)))) .$$

  LayerNorm operates on features at each token position only.
- Read the final valid position (`ANSWER`), predict one of `V0`–`V31`, and train with cross-entropy. Use right padding; do not select the final padded column by mistake.
- Reset the hidden state for each sample, each layer, and every forward pass. No state may carry over between batches. All operations must be causal.

Training settings:

- Start with AdamW, learning rate 10^{-3}, weight decay 0.01, batch size 64, 10 epochs, gradient clipping 1.0, and seed 42. Suggested initialization: a spread over 0.5–0.99 and gate bias zero. State any adjustments.
- For Questions 1–3, use identical data, backbone dimensions, batch size, optimizer settings, epoch budget, and seed for both models. Apply the same initialization rule to their shared parameter types.
- Select checkpoints by validation accuracy, breaking ties by lower validation loss. Test labels must not guide training or model selection.

2. **Implement and train an input-gated SSM.** (10%)
   The basic SSM uses fixed transition coefficients. In this variant, the current input controls how much the state is updated:

$$\widetilde{h}_t = a \odot h_{t-1} + Bx_t,$$
$$g_t = \sigma(W_g x_t + b_g),$$
$$h_t = (1 - g_t) \odot h_{t-1} + g_t \odot \widetilde{h}_t,$$
$$y_t = Ch_t + Dx_t.$$

   The gate has N entries, with $W_g \in \mathbb{R}^{N\times d}$ and $b_g \in \mathbb{R}^N$. This is a teaching version of a **Gated Diagonal SSM**, not a complete implementation of Mamba.

(a) Complete `gate_init`, `gate_compute`, and `gate_forward` in `GatedDiagonalSSM`. (5%)

(b) Plot training/validation loss and report the best validation accuracy and its epoch. (3%)

(c) **Implementation and discussion.** Explain how you compute the input-dependent gate and combine the old and candidate states. Describe what happens when g_t is near zero or one, and use your validation results to discuss whether gating helped. A correct implementation and justified analysis can receive full credit even if accuracy does not improve. (2%)

3. **Compare performance and computational cost.** (5%)
Use the selected checkpoints from Questions 1 and 2.

(a) **Measurements.** Fill in the table below using the measurement procedure specified here. (3%)

Metric	Basic SSM	Gated SSM
Trainable parameters		
Validation accuracy		
Training time (seconds/epoch)		
Inference throughput (samples/s)		
Peak GPU allocated memory (MiB)		

- Report the mean training-only epoch time, excluding validation and final plotting. Use the same device, precision, and batch size.
- For inference, use evaluation mode without gradients, warm up first, and synchronize GPU timing. The provided benchmark times repeated forward passes on a validation batch; report this scope and batch size.
- Report peak GPU allocated memory in MiB. If using CPU, write "N/A" for GPU memory. State hardware/software versions.

(b) **Implementation and discussion.** Explain how you measured runtime, throughput, and memory fairly, including warm-up and GPU synchronization when applicable. Compare the accuracy–cost trade-offs and give at least two possible reasons for the observed differences, considering parameter count and training randomness. Include representative failure cases. (2%)

NOTE: PyTorch and Matplotlib are required. Edit the five TODOs in the model cell. The data loader, training loop, curves, checkpoint saving, benchmark, and submission writer are provided. On Kaggle, set `DATA_DIR` to the actual competition input directory; save outputs in the working directory.

4. **Kaggle competition: selective memory challenge.** (25%)

(a) **One improvement and validation results.** Modify at least one factor: state dimension, depth, learning rate, training schedule, or prediction head. For Question 4 only, you may use task-consistent augmentation of `data/train.jsonl` (e.g., key/value renaming or valid query changes), updating labels consistently. Retain the specified basic or gated SSM updates and token-only inputs. The head may combine the final valid `ANSWER` representation with the observed query-key embedding. Keep the entire model below 10^6 trainable parameters. Do not train or augment with validation/test examples. Questions 1–3 must retain the prescribed backbone and matched settings. Compare your improvement with the original gated model on validation accuracy, parameter count, and runtime. (3%)

(b) **Competition submission.** Use one best validation-selected model to predict all 2,000 hidden test answers and upload `submission.csv` to the course Kaggle competition. The deadline is displayed on the competition page. (20%)

(c) **Implementation and discussion.** Explain what you changed, why you chose it, and how you implemented it. Use the validation comparison to discuss its effect, and explain how you selected the final checkpoint and generated the submission CSV without using test labels. A justified negative improvement result can receive full credit for this discussion. (2%)

Kaggle invitation:

https://www.kaggle.com/t/8c17aaa87189485593944446f43ea896c

Submission format:

```
id,answer
test_000001,V8
test_000002,V4
```

- Use exactly two columns, `id,answer`. Every official test ID must appear exactly once, with no extras. Answers must be exactly `V0`–`V31`. No probabilities or additional text are accepted.
- **Evaluation metric:** exact-match accuracy (higher is better).
- **Team name = your student ID.** Work individually. At most five submissions per day; select one final submission before the deadline. Include its CSV, Kaggle submission ID, checkpoint, and model configuration in your course submission.
- **Public/Private split:** 600 test examples (30%) are public, and 1,400 (70%) are private. The private score is revealed after the competition ends. Kaggle's final ranking uses the private leaderboard.
- **Competition grading (20 points):** Use the *same final selected submission* throughout. Baseline qualification uses $S = 0.30\,\mathrm{Acc}_{\mathrm{public}} + 0.70\,\mathrm{Acc}_{\mathrm{private}}$. The ordinary baseline is $B = 0.40050$ (40.05%), and the strong baseline is $T = 0.58350$ (58.35%). These thresholds use the weighted score S, not the Public score alone. The host benchmark rows are for reference and do not count as student entries.

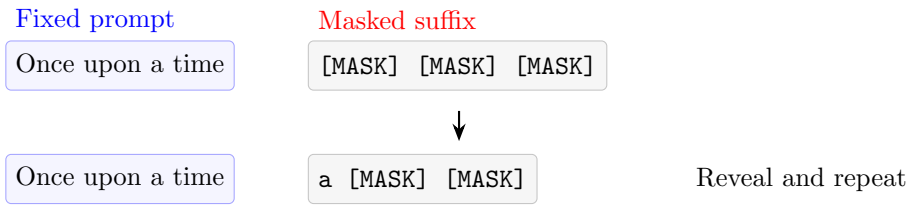
$$G_{\mathrm{competition}} = \begin{cases} 5S/B, & 0 \le S < B, \\ 5, & B \le S < T, \\ 10 + 10\dfrac{M-r}{M-1}, & S \ge T,\ M \ge 2, \\ 20, & S \ge T,\ M = 1. \end{cases}$$

  Here M counts eligible students reaching the strong baseline. Rank only these students by **Private accuracy**, highest first; $r = 1+$ the number with strictly higher Private accuracy. Tied scores receive the same rank and points. Exclude host benchmarks, non-students, and disqualified submissions.
  Thus the ordinary baseline earns 5 points; strong qualifiers earn 10–20 points by rank. If only one student qualifies, that student earns 20 points. Invalid or missing submissions earn 0. Qualification uses weighted accuracy; ranking uses Private accuracy, matching Kaggle's final ranking criterion.

2 Masked Diffusion Language Models (50%)

In this section, you will train a small Masked Diffusion Language Model (MDLM) from scratch on TinyStories. Imagine a storytelling assistant completing a prompt such as *"Once upon a time"*. Unlike an autoregressive model that predicts tokens from left to right, an MDLM generates text by repeatedly predicting and revealing masked tokens. You will investigate its denoising and generation behavior. The goal is to understand MDLM, not to produce perfect stories.

Dataset and model description:



Schematic only; generated examples must come from your trained model.

- Use the provided subset of 120,000 training stories and 1,000 validation stories. The validation set must not be used for training.

- Use the provided bidirectional Transformer: approximately 14M parameters, 6 layers, hidden size 384, and 6 attention heads. Use the provided 8K-token BPE tokenizer and maximum sequence length 128.
- Use one Google Colab T4 GPU, with a maximum GPU budget of 4 hours. Model weights must be randomly initialized; pretrained language-model weights are not allowed.

For an original sequence x_0, sample $t \sim \mathcal{U}(0, 1)$ and independently replace each non-padding token with `[MASK]` with probability t. Let x_t denote the corrupted sequence and M its masked positions. A simplified training objective is

$$\mathcal{L}_{\text{MDLM}} = -\mathbb{E}_{t,x_0,x_t}\left[\frac{1}{tL}\sum_{i\in M}\log p_\theta(x_0^i \mid x_t)\right],$$

where L is the number of non-padding tokens. During generation, start with a masked suffix and iteratively reveal predicted tokens.

Please follow the requirements below to implement your program:

1. Complete the following `TODO` sections in `hw2_2.ipynb`. The Transformer, tokenizer, data loader, and training loop are provided. (25%)
   (a) Implement `forward_masking`: sample masked positions using t, preserve padding tokens, and return the corrupted input and mask. Show examples for $t = 0.25, 0.50, 0.75$. (7%)
   (b) Implement `compute_mdlm_loss`: sample t, predict the original tokens, and compute cross-entropy only at masked positions, with the $1/t$ weighting above. Handle small t safely. (10%)
   (c) Implement `iterative_decode`: at each step, select the remaining masked positions with the highest prediction confidence and reveal their predicted tokens. Keep the prompt and previously revealed tokens fixed. (8%)
2. Train the provided model from random initialization on TinyStories. Plot the training loss and report the validation loss, optimizer steps, GPU training time, and model parameter count. Save the final checkpoint. Briefly describe whether the loss decreases and whether overfitting is evident. (10%)
3. Complete the following evaluation and analysis. (10%)
   (a) On the same held-out validation set, evaluate Masked Cross-Entropy (MCE) at masking ratios 25%, 50%, and 75%. Report a table or plot and explain the trend. Lower is better. (5%)
   (b) Generate 64-token sequences from the prompt *"Once upon a time"* using 4, 8, and 16 decoding steps. Show the generated text and measured decoding time for each setting. Compare fluency, repetition, and speed. (5%)
4. Answer the following questions briefly. (5%)
   (a) How does MDLM generation differ from autoregressive generation? (2%)
   (b) Why are different masking ratios used during training? (1%)
   (c) Why might a low reconstruction loss still produce repetitive or incoherent text? (2%)

NOTE:

- For a fixed masking ratio, MCE is the average negative log-probability of the original tokens at masked positions. Do not report exp(MCE) as standard autoregressive perplexity (PPL).
- Exact-match accuracy is optional because more than one word may be reasonable in a masked context. Grading emphasizes correct implementation and analysis, not a high generation score.

3 Rules

- In your course submission, submit two notebooks using the following filename format:
  - `hw2_<ProblemNumber>_<StudentID>.ipynb`, containing executable code, results, figures, metric values, and written discussion for each exercise (for example, `hw2_1_0123456.ipynb` and `hw2_2_0123456.ipynb`).
- Implementation will be graded based on
  - Completeness and algorithm correctness
  - Description of model design
  - Experimental analysis and reproducibility
  - Kaggle performance for Problem 1
- Only Python/PyTorch implementations are accepted. Report random seeds, training settings, and hardware/software versions. A separate PPT is not required.
- For **Problem 1**, retain the specified basic and gated SSM family. Self-attention, RNN/LSTM/GRU, pretrained models, external training data, and rule-based/dictionary retrieval in place of neural inference are not permitted. Effective trainable parameters must remain below 10^6.
- For **Problem 1**, only the token sequence may be a model input. Use `train.jsonl` for training and `val.jsonl` for model selection. Keep the provided split fixed. Do not train on test inputs or tune repeatedly to Public leaderboard feedback.
- For **Problem 1**, participate individually and use your student ID as the team name. Make at least one valid Kaggle submission. At most five submissions per day; select one final submission before the deadline. Include the final CSV, Kaggle submission ID, checkpoint, and model configuration in your course submission. Preserve the results of both required models.
- For **Problem 2**, use the provided dataset split, tokenizer, and model architecture. Train with randomly initialized weights, using one Colab T4 GPU within the 4-hour GPU budget. Do not use pretrained model weights or additional training data. Submit the trained checkpoint according to course platform instructions.
- Problem 1 contributes 50%: basic SSM 10%, gated SSM 10%, comparison 5%, improvement/analysis 5%, and Kaggle performance 20%. Problem 2 contributes 50%: implementation 25%, training 10%, evaluation/analysis 10%, and discussion 5%.
- DO NOT PLAGIARIZE. Follow the course academic integrity policy.